

Compiladors: Examen final de laboratori.

11 de juny de 2025

ATENCIÓ: Al Racó trobareu els jocs de proves i codi necessari per a fer l'examen.

ABANS DE COMENÇAR A FER RES, llegiu les instruccions del final de l'enunciat per veure com descarregar-lo i instal·lar-lo.

ATENCIÓ: Cal entregar l'examen en un fitxer **.tgz** pujat al Racó. **Llegiu les instruccions del final de l'enunciat** per veure com generar-lo.

PUNTUACIÓ: Els tres primers punts de la nota de laboratori s'obtenen amb els jocs de proves de la pràctica base. La resta s'obtenen superant els jocs de proves específics de l'examen.

La correcció és **automàtica**, a través dels jocs de proves d'aquest enunciat, més un conjunt addicional de jocs de proves privats.

IMPORTANT: L'examen consta de dos exercicis independents. Podeu fer-los en qualsevol ordre. Es recomana fer cada exercici incrementalment, resolent cada joc de proves abans de passar al següent.

1 Accés segur a arrays (3.5 punts)

Volem afegir a l'ASL la possibilitat d'especificar un valor alternatiu per si l'índex d'accés a un element d'un array cau fora dels límits de l'array. La sintaxi d'aquest accés especial és **A[expr1!expr2]** on **A** és un array, **expr1** és el valor de l'índex, i **expr2** és el valor a usar en cas que l'índex caigui fora de rang.

Per exemple:

```
1 func main()
2   var i,j,n: int
3   var x, y: float
4   var A, B: array[10] of float
5
6   i = 3;
7   x = y*n + A[i+1]; // acces a array sense control de limits
8   j = i*2;
9   y = B[j-1!2*x/3]; // acces amb control de limits.
10                      // Com que j=6, j-1 esta dins dels limits
11                      // i per tant el resultat es el valor B[5]
12   n=1;
13   y = B[2*j!3.1+n]; // acces amb control de limits.
14                      // Com que j=6, 2*j esta fora dels limits
15                      // i per tant el resultat es 3.1+n = 4.1
16
17 endfunc
```

Observeu que tant el valor de l'índex com el valor alternatiu són expressions el valor de les quals no necessàriament és conegut en temps de compilació, i per tant cal calcular-los en temps d'execució.

Joc de proves 1 (0.5 punts). Començarem modificant la gramàtica per afegir el nou accés a arrays. Es recomana crear una subregla (amb *rule label*) separada de la de l'accés normal a arrays. Només és necessari modificar l'accés a arrays en expressions, ja que no té sentit obtenir un valor alternatiu als accesos en *left_expr*. També caldrà afegir al TypeCheck les següents comprovacions:

- El tipus de la base es array (igual que amb l'accés normal a arrays).
- El tipus de l'index és enter (igual que amb l'accés normal a arrays).
- El tipus del valor alternatiu és el mateix que el tipus dels elements de l'array. Per ara no considerarem coercions.

Al mòdul auxiliar *SemErrors* proporcionat amb l'exàmen trobareu els errors específics d'aquesta nova expressió.

Un cop fets els canvis, el primer joc de proves:

```

1 func f(i : bool, b:array[10] of int) : int
2   var j,a : int
3   write (a + b);
4   a = a > 1;
5   b[j] = b[j!2.1*a[k+1]];
6   return a * b[a*2+1!b[3]>0];
7 endfunc
8
9 func main()
10  var i, j: int
11  var c1, c2: char
12  var x, y: float
13  var A: array[10] of int
14  var B: array[10] of float
15  var C: array[10] of char
16
17  j = B[i+1] + f(true, A) + f(x<=0, B);
18  x = y * A[j-1!22] + B[i+2!3.3] + A[i!x+1];
19  c1 = C[i+j!c2];
20  y = x - A[j-1!c1] + A[j+1!'@'] - B[x+2!y-1];
21
22  write y+x;
23 endfunc

```

hauria de produir els següents errors semàntics:

```

Line 3:11 error: Operator '+' with incompatible types.
Line 4:4 error: Assignment with incompatible types.
Line 5:13 error: Secure array access with incompatible expression.
Line 5:17 error: Array access to a non array operand.
Line 5:19 error: Identifier 'k' is undeclared.
Line 6:21 error: Secure array access with incompatible expression.
Line 17:4 error: Assignment with incompatible types.
Line 17:36 error: Parameter #2 with incompatible types in call to 'f'.
Line 18:39 error: Secure array access with incompatible expression.
Line 20:16 error: Secure array access with incompatible expression.
Line 20:28 error: Secure array access with incompatible expression.
Line 20:37 error: Array access with non integer index.

```

Joc de proves 2 (1 punt). A continuació començarem amb la generació de codi per als accessos segurs a arrays. En el primer joc de proves, els índexs sempre cauen dins dels límits i per tant, no és necessari encara que genereu codi per calcular el valor alternatiu.

Un cop fets els canvis, el segon joc de proves: <pre> 1 func f(M:array[10] of int) : int 2 var i,k,n,t : int 3 read t; 4 n=0; i=1; 5 while i<11 do 6 n = n + M[i-1!23*n-t+1]; 7 i = i+1; 8 endwhile 9 return n; 10 endfunc 11 12 func main() 13 var i, j: int 14 var x, y: float 15 var A: array[10] of int 16 var B: array[10] of float 17 18 i=0; 19 while i<10 do 20 A[i] = i*3; B[i] = i*2.4; 21 i = i+1; 22 endwhile 23 24 j = 1; 25 while j<10 do 26 x = j + A[j] + B[j]; 27 write x; write '\n'; 28 y = j + A[j-1!i+1] + B[j-1!x/2]; 29 write y; write '\n'; 30 j = j+1; 31 endwhile 32 33 write B[i-2] + f(A); write '\n'; 34 endfunc </pre>	amb l'entrada següent: <pre> 10 </pre> ha d'escriure: <pre> 6.4 1 12.8 7.4 19.2 13.8 25.6 20.2 32 26.6 38.4 33 44.8 39.4 51.2 45.8 57.6 52.2 154.2 </pre>
--	---

Joc de proves 3 (1 punt). En aquest joc de proves ja apareixen índexs fora dels límits i per tant, cal generar codi que comprovi si l'índex es correcte o no, generar codi per calcular el valor alternatiu, i combinar aquests codis en una estructura condicional que, segons calgui, obtingui el valor de la posició correcta o bé el valor alternatiu.

Un cop fets els canvis, el segon joc de proves: <pre> 1 func f(M:array[10] of int) : int 2 var i,k,n,t : int 3 read t; 4 n=0; i=1; 5 while i<11 do 6 n = n + M[i+1!23*n-t+1]; 7 i = i+1; 8 endwhile 9 return n; 10 endfunc 11 12 func main() 13 var i, j: int 14 var x, y: float 15 var A: array[10] of int 16 var B: array[10] of float 17 18 i=0; 19 while i<10 do 20 A[i] = i*3; B[i] = i*2.4; 21 i = i+1; 22 endwhile 23 24 j = 1; 25 while j<15 do 26 if j<10 then 27 x = j + A[j] + B[j]; 28 write "x="; write x; write '\n'; 29 endif 30 y = j + A[j-1!i+1] + B[j-1!x/2]; 31 write "y="; write y; write '\n'; 32 j = j+1; 33 endwhile 34 35 write B[i-2] + f(A); write '\n'; 36 endfunc </pre>	al llegir l'entrada següent: 14 ha de produir la sortida: x=6.4 y=1 x=12.8 y=7.4 x=19.2 y=13.8 x=25.6 y=20.2 x=32 y=26.6 x=38.4 y=33 x=44.8 y=39.4 x=51.2 y=45.8 x=57.6 y=52.2 y=58.6 y=50.8 y=51.8 y=52.8 y=53.8 75726.2
--	---

Joc de proves 4 (1 punt). Finalment, permetrem coercions del valor alternatiu. És a dir, si l'array es de reals i el valor alternatiu es un enter, el typeCheck no ha de donar error. També caldra convertir a real el valor alternatiu si és necessari.

Amb això, el següent joc de proves: <pre> 1 func f(M:array[10] of int) : int 2 var i,k,n,t : int 3 read t; 4 n=0; i=1; 5 while i<11 do 6 n = n + M[i+1!23*n-t+1]; 7 i = i+1; 8 endwhile 9 return n; 10 endfunc 11 12 func main() 13 var i, j: int 14 var x, y: float 15 var A: array[10] of int 16 var B: array[10] of float 17 18 i=0; 19 while i<10 do 20 A[i] = i*3; B[i] = i*2.4; 21 i = i+1; 22 endwhile 23 24 j = 1; 25 while j<15 do 26 if j<10 then 27 x = j + A[j] + B[j]; 28 write "x="; write x; write '\n'; 29 endif 30 y = j + A[j-1!i+1] + B[j-1!x/2]; 31 write "y="; write y; write '\n'; 32 j = j+1; 33 endwhile 34 35 write B[i-2!3*(i+j)] + f(A); write '\n'; 36 A[3]=A[5]*2; A[5]=A[0]+1; 37 write B[i+5!3*(i+j)] + f(A); write '\n'; 38 endfunc </pre>	al llegir l'entrada següent: <pre> 1234 5 </pre> ha d'escriure: <pre> x=6.4 y=1 x=12.8 y=7.4 x=19.2 y=13.8 x=25.6 y=20.2 x=32 y=26.6 x=38.4 y=33 x=44.8 y=39.4 x=51.2 y=45.8 x=57.6 y=52.2 y=58.6 y=50.8 y=51.8 y=52.8 y=53.8 45226.2 80039 </pre>
---	--

2 Instrucció zip (3.5 punts)

El segon exercici consisteix en dotar el llenguatge ASL de la instrucció **zip**, que omple un array a partir d'intercalar els elements de dos arrays donats.

La sintaxi d'aquesta instrucció és: **zip A and B into C** on A, B i C són arrays.

La instrucció *zip* omple l'array C de manera que

$$C_{2*i} = A_i; C_{2*i+1} = B_i; \forall i \in [0, \min\{|A|, |B|\} - 1]$$

És a dir, intercala un element de A i un de B fins que arriba al final de l'array més curt dels dos.

Per exemple:

```
1 func main()
2   var i,j,n: int
3   var x, y: float
4   var A,B: array[10] of int
5   var C: array[20] of int
6   var X: array[10] of char
7   var Y: array[4] of char
8   var Z: array[8] of char
9
10  i=0;
11  while i<10 do
12    A[i]=i; B[i]=9-i;
13    X[i]='a';
14    if i<4 then
15      Y[i]='b';
16    endif
17    i=i+1;
18  endwhile
19  // A = [0,1,2,3,4,5,6,7,8,9]
20  // B = [9,8,7,6,5,4,3,2,1,0]
21  // X = ['a','a','a','a','a','a','a','a','a','a']
22  // Y = ['b','b','b','b']
23  zip A and B into C;
24  // C = [0,9,1,8,2,7,3,6,4,5,5,4,6,3,7,2,8,1,9,0]
25  zip X and Y into Z;
26  // Z = ['a','b','a','b','a','b','a','b']
27 endfunc
```

Joc de proves 5 (0.5 punts). El primer pas és afegir la instrucció `zip` a la gramàtica, com una instrucció nova: `zip A and B into C`.

També caldrà fer les comprovacions semàntiques següents:

- A, B, i C són arrays.
- Els elements dels tres arrays són del mateix tipus.
- La mida de C és exactament el doble de la mida menor entre A i B.

Al mòdul auxiliar *SemErrors* proporcionat amb l'exàmen trobareu els errors específics d'aquesta nova instrucció.

Així, el primer joc de proves:

```
1 func ff(a: array[10] of int, c:array[20] of int)
2     zip a and a into c;
3 endfunc
4
5 func main()
6     var i,j,n: int
7     var x, y: float
8     var A,B: array[10] of int
9     var C: array[20] of int
10    var X: array[10] of char
11    var Y: array[4] of char
12    var Z: array[8] of char
13
14    zip A and C into B;
15    zip A and B into C;
16    zip A and X into B;
17    zip Z and Y into X;
18    zip X and Y into Z;
19    ff(B,C);
20    ff(Y,Z);
21    ff(C,C);
22 endfunc
```

genera els errors:

```
Line 14:2 error: Zip instruction with inappropriate or incompatible arguments.
Line 16:2 error: Zip instruction with inappropriate or incompatible arguments.
Line 17:2 error: Zip instruction with inappropriate or incompatible arguments.
Line 20:5 error: Parameter #1 with incompatible types in call to 'ff'.
Line 20:7 error: Parameter #2 with incompatible types in call to 'ff'.
Line 21:5 error: Parameter #1 with incompatible types in call to 'ff'.
```

Joc de proves 6 (1 punt). A continuació començarem amb la generació de codi. En aquest joc de proves, els arrays són de mida 1, pel que es pot superar el joc de proves sense necessitats de generar cap bucle.

<pre> 1 2 func ff(a: array[1] of int, c:array[2] of int) 3 var i:int 4 zip a and a into c; 5 write "a:"; write a[0]; write "\n"; 6 write "c:"; write c[0]; 7 write " "; write c[1]; write "\n"; 8 endfunc 9 10 func main() 11 var i,j,n: int 12 var x, y: float 13 var A,B: array[1] of int 14 var C: array[2] of int 15 var X: array[1] of char 16 var Y: array[1] of char 17 var Z: array[2] of char 18 19 read A[0]; read B[0]; 20 read X[0]; read Y[0]; 21 22 zip A and B into C; 23 write "C:"; write C[0]; 24 write " "; write C[1]; write "\n"; 25 26 zip X and Y into Z; 27 write "Z:"; write Z[0]; 28 write " "; write Z[1]; write "\n"; 29 30 ff(B,C); 31 write "C2:"; write C[0]; 32 write " "; write C[1]; write "\n"; 33 endfunc </pre>	al processar l'entrada: <pre> 20 30 f g </pre> ha de generar la sortida: <pre> C:20 30 Z:f g a:30 c:30 30 C2:30 30 </pre>
--	---

Joc de proves 7 (1 punt). El següent pas consistirà en generar codi generic, per arrays de qualsevol mida.

El codi d'aquest joc de proves:

```

1 func ff(a: array[10] of int, c:array[20] of int)
2   var i:int
3   zip a and a into c;
4   write "a:"; i=0;
5   while i<10 do
6     write " "; write a[i]; i = i+1;
7   endwhile
8   write "\n";
9   write "c:"; i=0;
10  while i<20 do
11    write " "; write c[i]; i = i+1;
12    if i==10 then write "\n "; endif
13  endwhile
14  write "\n";
15 endfunc
16
17 func main()
18   var i,j,n: int
19   var x, y: float
20   var A,B: array[10] of int
21   var C: array[20] of int
22   var X: array[10] of char
23   var Y: array[4] of char
24   var Z: array[8] of char
25
26   i=0;
27   while i<10 do
28     read A[i]; read B[i]; read X[i];
29     if i<4 then read Y[i]; endif
30     i = i+1;
31   endwhile
32
33   zip A and B into C;
34   write "C:"; i=0;
35   while i<20 do
36     write " "; write C[i]; i = i+1;
37     if i==10 then write "\n "; endif
38   endwhile
39   write "\n";
40
41   zip X and Y into Z;
42   write "Z:"; i=0;
43   while i<8 do
44     write " "; write Z[i]; i = i+1;
45   endwhile
46   write "\n";
47
48   ff(B,C);
49   write "C2:"; i=0;
50   while i<20 do
51     write " "; write C[i]; i = i+1;
52     if i==10 then write "\n "; endif
53   endwhile
54   write "\n";
55 endfunc

```

al llegir l'entrada:

```

0 100 a W
1  90 b X
2  80 c Y
3  70 d Z
4  60 e
5  50 f
6  40 g
7  30 h
8  20 i
9  10 j

```

ha de produir la sortida:

```

C: 0 100 1 90 2 80 3 70 4 60
   5 50 6 40 7 30 8 20 9 10
Z: a W b X c Y d Z
a: 100 90 80 70 60 50 40 30 20 10
c: 100 100 90 90 80 80 70 70 60 60
   50 50 40 40 30 30 20 20 10 10
C2: 100 100 90 90 80 80 70 70 60 60
     50 50 40 40 30 30 20 20 10 10

```

Joc de proves 8 (1 punt). Finalment, tractarem les coercions: si els elements de l'array de destí són reals, i els elements d'algun dels arrays d'origen són enters, cal no donar error al TypeCheck, i fer les conversions de tipus adequades al moment de copiar els elements.

El codi d'aquest joc de proves:

```

1 func ff(a: array[10] of int, c:array[20] of float)
2   var i:int
3   zip a and a into c;
4   write "a:"; i=0;
5   while i<10 do
6     write " "; write a[i]; i = i+1;
7   endwhile
8   write "\n";
9   write "c:"; i=0;
10  while i<20 do
11    write " "; write c[i]; i = i+1;
12    if i==10 then write "\n "; endif
13  endwhile
14  write "\n";
15 endfunc
16
17 func main()
18   var i,j,n: int
19   var x, y: float
20   var A: array[10] of int
21   var B: array[15] of float
22   var C: array[20] of float
23
24   i=0;
25   while i<15 do
26     read B[i];
27     if i<10 then read A[i]; endif
28     i = i+1;
29   endwhile
30
31   zip A and B into C;
32   write "C:"; i=0;
33   while i<20 do
34     write " "; write C[i]; i = i+1;
35     if i==10 then write "\n "; endif
36   endwhile
37   write "\n";
38
39   ff(A,C);
40   write "C2:"; i=0;
41   while i<20 do
42     write " "; write C[i]; i = i+1;
43     if i==10 then write "\n "; endif
44   endwhile
45   write "\n";
46
47 endfunc

```

al llegir l'entrada següent:

```

0 100
1 90
2 80
3 70
4 60
5 50
6 40
7 30
8 20
9 10
10
11
12
13
14

```

ha de generar la sortida:

```

C: 100 0 90 1 80 2 70 3 60 4
   50 5 40 6 30 7 20 8 10 9
a: 100 90 80 70 60 50 40 30 20 10
c: 100 100 90 90 80 80 70 70 60 60
   50 50 40 40 30 30 20 20 10 10
C2: 100 100 90 90 80 80 70 70 60 60
    50 50 40 40 30 30 20 20 10 10

```

Informació important

FITXERS PER A L'EXAMEN: Al Racó (`examens.fib.upc.edu`) trobareu un fitxer `examen.tgz` amb el següent contingut:

- `final-lab-CL-2025.pdf`: Aquest document, amb l'enunciat i les instruccions.
- `jps`: Subdirectori amb jocs de proves (`jp_chkt_XX.asl`) i `jp_genc_YY.asl`), i la seva corresponent sortida esperada (`jp_chkt_XX.err`) per als jocs de proves de validació semàntica, `jp_genc_YY.in/.out` per als jocs de proves de generació de codi). En els JPs de generació, no es compara el codi generat, sinó la sortida que produeix la tVM en executar-lo.
- `common`: Subdirectori amb el mòdul auxiliar `SemErrors` ampliat amb el codi necessari per a l'examen.
- `evalua.sh`: Script que executa tots els jocs de proves i diu si se superen o no.
- `empaqueta.sh`: Script que crea un fitxer `examen-nom.cognom.tgz` amb la vostra solució. Aquest és el fitxer que cal pujar al Racó.

PASSOS A SEGUIR:

- Feu una còpia de les carpetes `asl`, `common` i `tvm` de la vostra pràctica a un directori `examen`.

```
mkdir examen
cp -r practica/asl practica/common practica/tvm examen/
```

- Canvieu al nou directori `examen`, i descomprimiu-hi el fitxer `examen.tgz` del Racó:

```
cd examen
tar -xzf examen.tgz
```

Això extreurà el contingut del paquet, **afegint** al vostre directori `examen` els fitxers llistats anteriorment.

IMPORTANT: Feu-ho en l'ordre especificat (primer una còpia de la vostra pràctica i després descomprimir el `.tgz`). Fer-ho en l'ordre invers causarà que us falti codi necessari a `common` i que els JPs no siguin els adequats.

- Trebal·leu normalment a la carpeta `examen/asl`.

```
cd asl
make pristine
make antlr
make -j 10
...
```

(Si la compilació és lenta per sobrecàrrega del servidor, podeu executar l'script `fast-make.sh`)

- Per executar tots els jocs de proves i veure si els passeu, executeu `../evalua.sh`.
- Per veure les diferències entre la sortida del vostre `asl` i la sortida esperada en un joc de proves concret de type check, podeu fer:

```
./asl ../jps/jp_chkt_XX.asl | diff -y - ../jps/jp_chkt_XX.err
```

(Podeu ignorar la línia "There are semantic errors: no code generated")
- Per veure les diferències entre la sortida del vostre `asl` i la sortida esperada en un joc de proves concret de generació de codi, podeu fer:

```
./asl ../jps/jp_genc_XX.asl > jp_XX.t
../tvm/tvm jp_XX.t < ../jps/jp_genc_XX.in | diff -y - ../jps/jp_genc_XX.out
```
- Executeu `../empaqueta.sh` per crear el fitxer d'entrega `../examen-USERNAME.tgz` que cal pujar al Racó. Els paquets creats sense usar aquest script seran qualificats com **NO PRESENTAT**.